Document number: P2838R0
Audience: SG21

Ville Voutilainen
2023-03-22

Unconditional contract violation handling of any kind is a serious problem

Abstract

This paper explains why we shouldn't have hard-coded Eval_and_abort and Eval_and_throw modes in the Contracts MVP, and further explains why we should have a violation handler instead. For those in a serious hurry, go read Joshua Berne's paper P2811 - Contract-Violation Handlers.

This paper's title is hinting a relationship to Bjarne's paper P2698R0 - Unconditional termination is a serious problem. That is not an accident - this paper provides an additional perspective on that paper's findings.

In a nutshell, C asserts and the Contracts MVP modes as previously proposed have a well-known problem - their operation can't be controlled without recompilation. For large-scale applications, that's a problem, because

1. Recompiling all code, all libraries that your application uses, and all libraries those libraries use, is costly, and if we can avoid that cost when someone needs a different way to handle contract violations, we should avoid it.
2. Shipping libraries compiled in multiple different modes is costly, too.
3. In some cases, you'll end up in a situation where you can't recompile all code, for whatever reason - either it's too costly, or you don't even have all the code for you to recompile.

This is much more than just a technicality that we could hope to skate around with gradual changes. It directly affects how contracts in C++ can be used, and how the code using them needs to be packaged and deployed, and which use cases require recompilation or making multiple binaries available, and which ones don't. It affects both library vendors and library users, and has a big impact on how to ship binary libraries, regardless of whether they are open-source or proprietary.

So what exactly is proposed here?

I'm proposing that we adopt P2811 - Contract-Violation Handlers into the Contracts MVP for C++26.

The goal of getting the end result of Eval_and_abort is achieved by P2811 directly - it specifies that if a violation handler returns normally, the program is aborted. We can also support Eval_and_throw by actually specifying that the violation handler is replaceable, and allowing users who want it to provide their own throwing violation handler. Or, as an alternative, a handler that puts the thread of execution to sleep, or makes it spin endlessly. Such handlers do not return normally, so the program is not aborted, and we achieve the goal of allowing alternative violation handler behaviors.

Does the MVP design preclude adding a violation handler later?

Well, from a pure specification standpoint, of course not. Few things preclude others that way.

However, it does preclude achieving the goal that we (apparently, for some values of "we") want. If implementations arise that don't provide a violation handler as a QoI matter despite the specification not requiring one, and bake/hard-code the violation handler semantics into compiled translation units, then we can't backtrack on that choice later. So we end up in the same situation as we end up with C asserts - the only thing we can do is turn them on or off via recompilation. And the only thing a violation can do is what it was compiled to do, which is then only ever one single thing, and there's nothing users can do to change that when linking their final program.

Okay, so what is the problem?

The problem is that hard-coded violation handler semantics don't scale. For Bjarne's concerns, if there's any translation unit anywhere in a program that terminates on contract violation, that translation unit needs to be recompiled. If another user doesn't want contract violations to throw exceptions, the same problem occurs, any translation unit that has been compiled with a throwing violation handler strategy needs to be recompiled.

Any moderately complex desktop application links to dozens of dynamic C++ libraries. Any moderately complex embedded application, like an In-Vehicle Infotainment system of your car links to more.

Not all of those libraries are open-source, so you can't always recompile all of them. We will then face the reality that the library vendor will need to provide a handful of different configurations of the library. Currently, it's pushing it to make such vendors provide more than two different configurations (an optimized release build and a debug build where various instrumentations may be enabled as a bunch, in addition to having asserts enabled and debug information present).

Even during development in the workplace of a programmer, having to do a mass rebuild to change violation handling semantics is a very heavy operation, it can take multiple hours, and slows down development significantly.

And we can easily do better. Just specify that contract checks are either completely disabled, or that they are enforced, and in the latter case, they invoke a violation handler. We have a shipping implementation of a violation handler that's user-replaceable. Using this approach, an "owner of main()" can decide what the contract violation handling semantics are for the program at hand, including *all* of its library dependencies, as long as they have compiled to invoke the violation handler instead of performing some specific violation handling action directly. Changing the chosen strategy doesn't require recompilation of *anything*, all it requires is relinking your program with a different readily-built violation handler.

A soundbite (or a design principle) version of this

It is often so that common and highly-generic libraries are at the bottom of a library dependency chain. The standard library is an excellent example. Boost libraries are another. On top of those libraries, more domain-specific libraries are built, going to the actual translation units that are not "library code" by any definition of the word - they are "application code", and they are the most domain-specific part of the dependency chain.

But it's very often so that in those last parts resides the knowledge of what the users want and what the application domain needs - including what sort of violation handling strategies are acceptable. The further away from the actual application a library is, the less appropriate it is for that library to choose a violation handling strategy.

The MVP approach doesn't support this sort of layering. A (replaceable) violation handler that all contract checking statements call does.

Can we rely on Vendor QoI to solve this problem?

The simple answer is No. We can't "rely on QoI". That's not a thing, to rely on things that are unspecified or underspecified.

We do have an experimental contracts implementation that already ships a replaceable violation handler, so for that implementation to be amended to implement a C++26 contracts facility, sure, we can perhaps expect that it will use a violation handler anyway to implement our MVP semantics, even though it's not required to.

But we arguably know what we want, so let's specify what we want, and let's not leave it up to rainy wishes and QoI.

Is this more complicated than what's in the MVP?

To a fair extent.. NO. For the MVP, we desperately try to avoid the problem of dealing with different translation units compiled in different contract-checking modes and how those interact.

This suggested approach simplifies the picture. It's not difficult at all to link a TU with contract checking turned off with a TU that has contract checking enforced, or vice versa. Linking a TU with contract checking enforced with another TU that has contract checking enforced is similarly trivial. And now we can decide how both of them handle violations, at a program-wide level. This is a part of the C++2a contracts design that was useful and right, we should reuse it, and adopt it.

Does this approach preclude finer-grained (like TU-specific or even case-by-case) contract semantics?

No, I don't think it does. But to provide a scalable starting point, we should start with an approach where violation handling semantics can be altered without having to recompile the whole world, and after that, probably post C++26, consider approaches that go into that other direction.

This also applies to possible desires to say "I KNOW what I want, I want Eval_and_abort, please just hard-code that into the object code". There's no fundamental reason to prevent users from doing that, but that's a lesser goal because *that* can be provided by QoI at first, and as a guaranteed facility later, but we can't reasonably start from a finer-grained control and expect to rein it in later with a scalable approach; that doesn't work if the horses are already out of the barn. And for e.g. C asserts, they are.